

Documentação Oficial da Solução

Ozzy - Task Manager

Versão: 2.0 — MVP

Plataforma: [Bubble.io](#) — Plano Gratuito

1. Contextualização do Desafio

1.1 Contextualização do Desafio

O **Ozzy - Task Manager** é uma aplicação web desenvolvida para facilitar o gerenciamento colaborativo de projetos e tarefas.

A solução foi concebida para atender à necessidade de centralizar o acompanhamento de atividades, permitindo que usuários criem projetos, distribuam tarefas, acompanhem prazos, atualizem status, registrem comentários e acompanhem o histórico das ações realizadas.

Problema

Em processos tradicionais de gerenciamento de atividades, informações podem ficar distribuídas entre diferentes ferramentas, planilhas, mensagens e controles manuais. Essa fragmentação dificulta o acompanhamento das responsabilidades, dos prazos e da evolução das tarefas.

O desafio consiste em disponibilizar uma solução simples e centralizada que permita:

- Organizar Projetos E Tarefas Em Um Único Ambiente.
- Definir Responsáveis Para Cada Tarefa.
- Acompanhar Status, Prioridades E Prazos.
- Registrar Comentários E Interações.
- Disponibilizar Notificações Aos Usuários.

- Manter Um Histórico Das Principais Ações.
- Facilitar Acompanhamento E Manutenção Da Operação.

Público-Alvo

A aplicação é destinada principalmente a equipes e usuários que necessitam organizar atividades de forma colaborativa, acompanhando projetos e tarefas em um ambiente centralizado.

O MVP contempla dois perfis de usuário:

- Administrator.
- Member.

Benefícios Esperados

A solução busca proporcionar:

- Maior Organização Das Atividades.
 - Maior Visibilidade Sobre Projetos E Tarefas.
 - Melhor Controle De Responsabilidades.
 - Redução Da Dependência De Controles Manuais.
 - Maior Rastreabilidade Das Alterações.
 - Comunicação Centralizada Nas Tarefas.
 - Facilidade De Uso E Manutenção.
 - Base Estrutural Para Futuras Expansões.
-

2. Justificativa Pelo Uso De No-Code/Low-Code

2.1 Escolha Da Plataforma

O **Bubble.io** foi escolhido como plataforma de desenvolvimento devido à necessidade de construir rapidamente um MVP funcional utilizando uma abordagem No-Code/Low-Code.

A solução utiliza recursos nativos da plataforma, incluindo:

- Banco De Dados Nativo.

- Sistema De Autenticação.
- Workflows.
- Reusable Elements.
- Option Sets.
- Privacy Rules.
- Gerenciamento De Interfaces.
- Ambiente De Desenvolvimento E Produção.

A utilização desses recursos reduz a necessidade de desenvolvimento de infraestrutura própria e permite concentrar os esforços na implementação das regras de negócio e da experiência do usuário.

2.2 Vantagens Em Relação Ao Desenvolvimento Tradicional

A utilização do Bubble proporciona ganhos importantes para o contexto do MVP:

- Menor Tempo De Desenvolvimento.
- Menor Necessidade De Código Manual.
- Redução Da Complexidade De Infraestrutura.
- Desenvolvimento Integrado De Front-End E Back-End.
- Banco De Dados Integrado À Aplicação.
- Implementação Rápida De Workflows.
- Facilidade Para Alterar A Interface.
- Maior Velocidade Na Validação Do Produto.

Em um desenvolvimento tradicional, seria necessário estruturar separadamente aplicação, banco de dados, autenticação, APIs, infraestrutura e processos de deploy.

No Bubble, grande parte desses recursos está integrada à própria plataforma.

2.3 Limitações

Apesar dos ganhos de produtividade, a abordagem No-Code apresenta algumas limitações:

- Dependência Da Plataforma Bubble.
- Limitações Do Plano Gratuito.
- Menor Controle Sobre A Infraestrutura.
- Dependência Dos Recursos Disponibilizados Pela Plataforma.
- Possíveis Limitações Para Processamentos Muito Complexos.

- Necessidade De Adequação Da Arquitetura Às Características Do Bubble.

Essas limitações foram consideradas durante o planejamento e a arquitetura do MVP.

2.4 Ganhos De Produtividade, Custo E Tempo

A utilização do Bubble permite reduzir significativamente o esforço necessário para construir a primeira versão da aplicação.

O principal ganho esperado está na capacidade de validar rapidamente a solução sem a necessidade de desenvolver uma infraestrutura completa desde o início.

A estratégia permite:

- Reduzir O Tempo Entre Ideia E MVP.
 - Reduzir O Custo Inicial De Desenvolvimento.
 - Facilitar Alterações Durante A Validação.
 - Permitir Evolução Incremental.
 - Concentrar O Desenvolvimento Nas Regras De Negócio.
 - Validar A Solução Antes De Investimentos Maiores Em Infraestrutura.
-

3. Modelagem De Dados

3.1 Estratégia

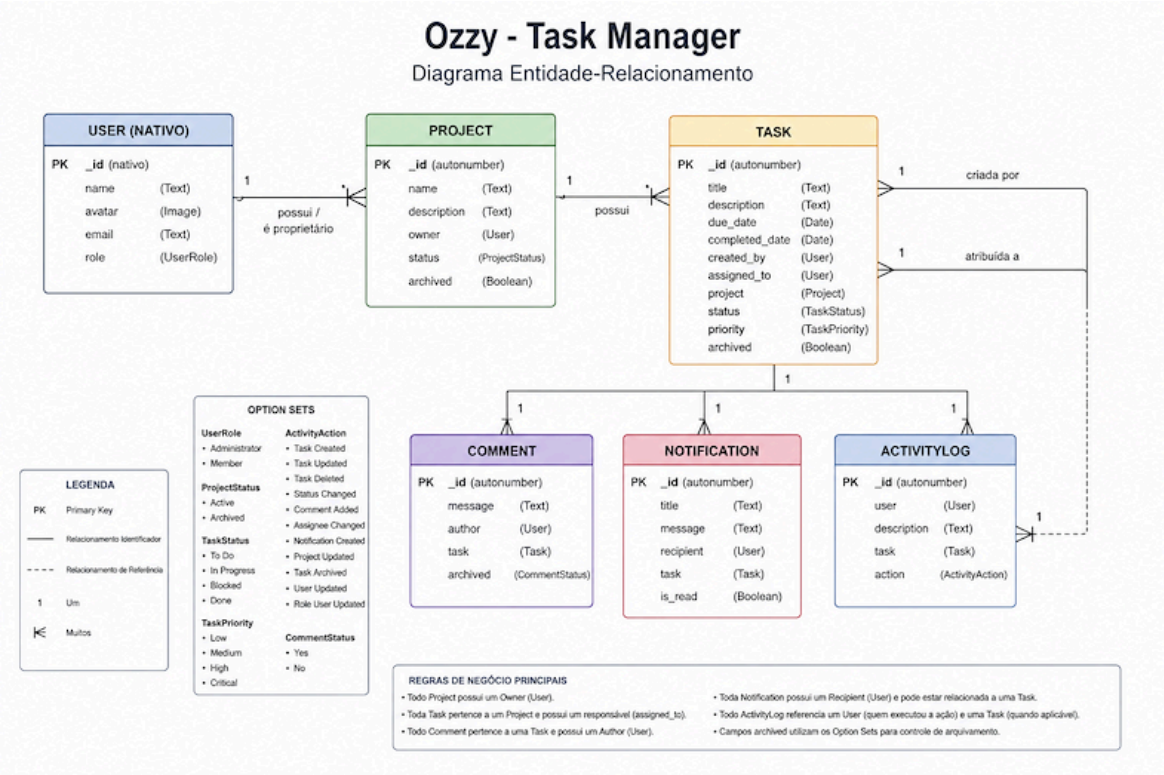
A modelagem utiliza exclusivamente os **Data Types nativos do Bubble**, priorizando referências entre entidades e evitando duplicação de informações.

O tipo **User** nativo do Bubble é utilizado para autenticação e identificação dos usuários.

As principais entidades são:

- User.
- Project.
- Task.
- Comment.
- Notification.
- ActivityLog.

Diagrama de Entidade-Relacionamento:



3.2 User

Representa os usuários da plataforma.

Campos

Campo	Tipo
name	Text
avatar	Image
role	UserRole
email	Text

O campo **email** também está relacionado ao mecanismo nativo de autenticação do Bubble.

Responsabilidades

- Autenticação.

- Proprietário De Projetos.
 - Criador De Tarefas.
 - Responsável Por Tarefas.
 - Autor De Comentários.
 - Destinatário De Notificações.
 - Autor De Registros De Atividades.
-

3.3 Project

Representa um projeto e funciona como agrupador lógico das tarefas.

Campos

Campo	Tipo
name	Text
owner	User
archived	Boolean
description	Text
status	ProjectStatus

Responsabilidades

- Organizar Tarefas.
 - Centralizar Informações Do Projeto.
 - Definir O Proprietário.
 - Controlar O Estado Do Projeto.
-

3.4 Task

É a entidade operacional central da aplicação.

Campos

Campo	Tipo
-------	------

title	Text
description	Text
due_date	Date
completed_date	Date
archived	Boolean
created_by	User
assigned_to	User
project	Project
status	TaskStatus
priority	TaskPriority

Responsabilidades

- Representar Atividades.
 - Vincular A Atividade A Um Projeto.
 - Definir O Responsável.
 - Controlar Status.
 - Controlar Prioridade.
 - Controlar Prazo.
 - Receber Comentários.
 - Originar Notificações.
 - Servir Como Referência Para O Histórico.
-

3.5 Comment

Representa comentários associados às tarefas.

Campos

Campo	Tipo
author	User

message	Text
task	Task
archived	CommentStatus

O campo `archived` utiliza o Option Set `CommentStatus` como mecanismo de controle de arquivamento.

Responsabilidades

- Registrar Interações Sobre Uma Tarefa.
 - Identificar O Autor Do Comentário.
 - Manter O Comentário Associado À Tarefa.
-

3.6 Notification

Representa notificações internas destinadas aos usuários.

Campos

Campo	Tipo
title	Text
message	Text
recipient	User
is_read	Boolean
task	Task

Responsabilidades

- Informar Alterações Relevantes.
 - Informar Atribuição De Tarefas.
 - Informar Comentários.
 - Informar Alterações De Status.
 - Permitir Controle De Leitura.
-

3.7 ActivityLog

Responsável pelo registro das principais ações realizadas na aplicação.

Campos

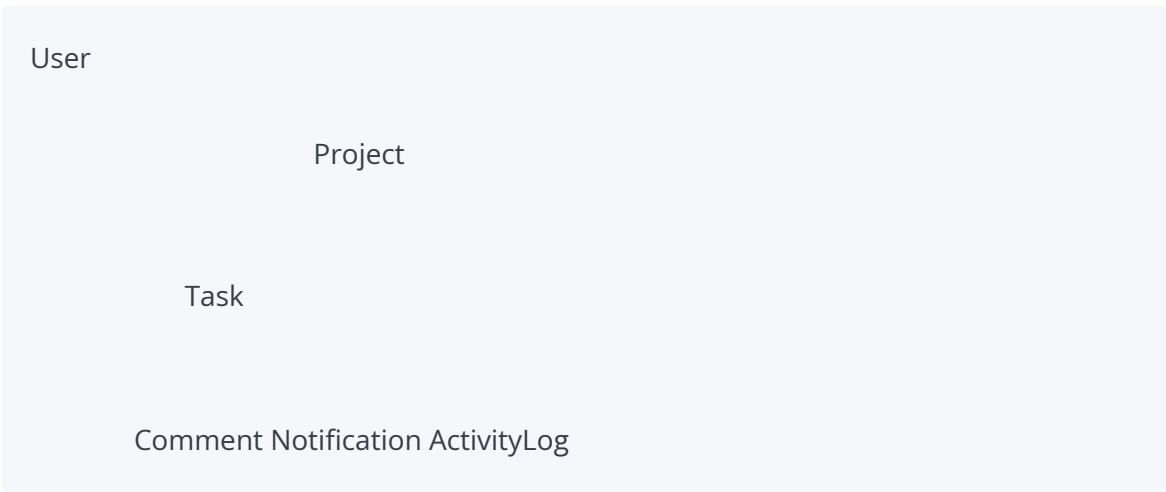
Campo	Tipo
user	User
description	Text
task	Task
action	ActivityAction

Responsabilidades

- Registrar Alterações Importantes.
 - Manter Histórico Das Tarefas.
 - Apoiar Auditoria.
 - Permitir Visualização Da Evolução Das Atividades.
-

3.8 Relacionamentos

Os principais relacionamentos são:



Relacionamentos Principais

- Um User Pode Possuir Vários Projects.

- Um Project Possui Várias Tasks.
 - Cada Task Pertence A Um Project.
 - Um User Pode Criar Várias Tasks.
 - Um User Pode Ser Responsável Por Várias Tasks.
 - Uma Task Pode Possuir Vários Comments.
 - Uma Task Pode Originar Várias Notifications.
 - Uma Task Pode Possuir Vários ActivityLogs.
 - Um User Pode Gerar Vários ActivityLogs.
-

3.9 Option Sets

Os valores fixos da aplicação são controlados por Option Sets.

UserRole

- Administrator.
- Member.

ProjectStatus

- Active.
- Archived.

TaskStatus

- To Do.
- In Progress.
- Blocked.
- Done.

TaskPriority

- Low.
- Medium.
- High.
- Critical.

ActivityAction

- Task Created.

- Task Updated.
- Task Deleted.
- Status Changed.
- Comment Added.
- Assignee Changed.
- Notification Created.
- Project Updated.
- Task Archived.
- User Updated.
- Role User Updated.

CommentStatus

- Yes.
- No.

O `CommentStatus` é utilizado como flag de arquivamento dos comentários.

3.10 Regras De Privacidade

As Privacy Rules controlam o acesso aos registros.

User

- Everyone Pode Visualizar E Pesquisar Campos Públicos.
- O Próprio Usuário Possui Acesso Aos Seus Dados E Anexos Conforme Configuração.

Project

- O Owner Possui Acesso Completo Ao Registro.
- Everyone Possui Apenas Acesso De Leitura E Pesquisa Conforme Configuração.

Task

- O Creator Possui Controle Completo Sobre A Tarefa.
- O Assignee Pode Alterar Status E Completed Date.
- Demais Usuários Possuem Apenas Acesso De Leitura E Pesquisa Conforme Configuração.

Comment

- O Author Possui Controle Completo.
- Demais Usuários Possuem Apenas Acesso De Leitura E Pesquisa Conforme Configuração.

Notification

- O Registro Não É Disponibilizado Para Consulta Geral.
- Apenas O Recipient Pode Visualizar A Notificação E Alterar `is_read` .

ActivityLog

- Usuários Autenticados Possuem Acesso De Leitura E Pesquisa Conforme As Regras Configuradas.
-

4. Lógica E Workflows

4.1 Estratégia

A lógica de negócio é implementada por meio dos **Workflows do Bubble**.

As páginas são responsáveis principalmente pela interação com o usuário, enquanto os Workflows executam:

- Validações.
- Alterações No Banco.
- Controle De Permissões.
- Registro De Histórico.
- Criação De Notificações.
- Atualização Da Interface.

O fluxo geral segue:

Evento

Validação

Permissão

Regra De Negócio

Persistência

ActivityLog

Notification

Atualização Da Interface

4.2 Autenticação

Cadastro

O Workflow de cadastro:

1. Apenas um administrador pode cadastrar um novo usuário.

Login

O Workflow:

1. Valida As Credenciais.
2. Autentica O Usuário.
3. Cria A Sessão.
4. Redireciona Para O Dashboard.

Logout

O Workflow:

1. Encerra A Sessão.
2. Redireciona Para Login.

Recuperação De Senha

Utiliza O Mecanismo Nativo De Recuperação De Senha Do Bubble.

4.3 Projetos

Criar Projeto

O Workflow:

1. Valida O Nome.
2. Cria O Project.
3. Define O Current User Como Owner.
4. Define Status Inicial Como Active.
5. Define `archived = no` .
6. Atualiza A Interface.

Editar Projeto

Permite atualizar informações do projeto conforme as permissões do Owner.

Arquivar Projeto

O Workflow:

1. Define `archived = yes` .
 2. Define `status = Archived` .
 3. Atualiza A Listagem.
-

4.4 Tarefas

Criar Tarefa

A criação exige:

- Projeto.
- Título.
- Responsável.

Após a criação:

1. O Task É Persistido.
2. O Criador É Registrado Em `created_by` .
3. O Responsável É Registrado Em `assigned_to` .
4. O Projeto É Associado.

5. O Status E A Prioridade São Definidos.
6. Um ActivityLog É Criado.
7. Uma Notification É Criada Para O Responsável.
8. A Interface É Atualizada.

Editar Tarefa

Permite atualização dos dados conforme as permissões definidas.

Quando aplicável:

- Um ActivityLog É Criado.
- Uma Notification É Criada.

Excluir Tarefa

A exclusão é permitida ao criador da tarefa.

Após a confirmação:

1. A Permissão É Validada.
2. A Tarefa É Excluída.
3. A Ação É Registrada No ActivityLog.

Alterar Status

Quando o status é alterado:

1. O Novo Status É Persistido.
2. Se O Status For Done, `completed_date` Recebe A Data Atual.
3. Um ActivityLog É Criado.
4. Uma Notification É Criada Quando Aplicável.
5. A Interface É Atualizada.

Alterar Responsável

Quando o responsável é alterado:

1. `assigned_to` É Atualizado.
 2. Um ActivityLog É Criado.
 3. Uma Notification É Enviada Ao Novo Responsável.
-

4.5 Comentários

Adicionar Comentário

O Workflow:

1. Valida A Mensagem.
2. Cria O Comment.
3. Associa O Autor.
4. Associa A Tarefa.
5. Cria Um ActivityLog.
6. Cria Uma Notification Para O Responsável.
7. Atualiza A Lista De Comentários.

Editar Comentário

O autor pode atualizar o comentário conforme as regras de privacidade.

Excluir Comentário

O autor pode excluir o comentário conforme as regras de privacidade.

4.6 Notificações

Criar Notificação

A criação de notificações é utilizada pelos demais Workflows.

Os principais dados são:

- Title.
- Message.
- Recipient.
- Task.
- Is Read.

Marcar Como Lida

Quando o usuário acessa uma notificação:

1. O Recipient É Validado.
 2. `is_read` É Atualizado Para Yes.
 3. A Interface É Atualizada.
-

4.7 ActivityLog

O ActivityLog é utilizado para registrar ações importantes.

Entre as ações registradas estão:

- Task Created.
- Task Updated.
- Task Deleted.
- Status Changed.
- Comment Added.
- Assignee Changed.
- Notification Created.
- Project Updated.
- Task Archived.
- User Updated.
- Role User Updated.

O registro deve identificar o usuário responsável pela ação e, quando aplicável, a tarefa relacionada.

4.8 Perfil

O usuário autenticado pode atualizar:

- Name.
- Avatar.

A alteração é realizada sobre o próprio registro do usuário.

5. Publicação E Manutenção

5.1 Estratégia De Ambientes

O desenvolvimento será realizado utilizando a separação de ambientes disponibilizada pelo Bubble.

Development

Utilizado para:

- Desenvolvimento.
- Configuração.
- Alterações Estruturais.
- Testes.
- Validação De Workflows.
- Validação De Privacy Rules.

Live

Utilizado para:

- Disponibilizar A Aplicação Aos Usuários.
- Executar A Versão Estável Do MVP.
- Receber Alterações Após Validação.

As alterações deverão ser validadas no ambiente de desenvolvimento antes de serem disponibilizadas no ambiente Live.

5.2 Processo De Publicação

O processo recomendado é:

Alteração

Development

Testes

Validação

Revisão

Publicação

Antes da publicação devem ser verificados:

- Funcionalidades.
 - Workflows.
 - Privacy Rules.
 - Banco De Dados.
 - Interface.
 - Responsividade.
 - Documentação.
-

5.3 Manutenção Contínua

A manutenção deverá ser realizada de forma controlada.

Antes de alterações estruturais importantes:

- Revisar O Impacto Na Modelagem.
 - Revisar Os Workflows Dependentes.
 - Revisar As Privacy Rules.
 - Revisar Os Option Sets.
 - Atualizar A Documentação.
 - Executar Testes De Regressão.
-

5.4 Backup E Recuperação

As informações da aplicação devem ser protegidas por meio dos mecanismos de versionamento e recuperação disponibilizados pelo Bubble.

Para alterações críticas, recomenda-se manter uma versão estável identificável antes da aplicação das mudanças.

O objetivo é permitir:

- Identificação Da Versão Estável.
 - Recuperação Em Caso De Falha.
 - Comparação Entre Alterações.
 - Redução Do Risco Durante Mudanças Estruturais.
-

5.5 Métricas E Monitoramento

A manutenção deverá acompanhar indicadores relacionados ao funcionamento da aplicação.

Entre os principais pontos:

- Erros De Workflows.
- Falhas De Autenticação.
- Tempo De Carregamento Das Páginas.
- Comportamento Do Dashboard.
- Funcionamento Das Listagens.
- Volume De Registros.
- Problemas De Permissão.
- Erros Reportados Pelos Usuários.

A análise desses indicadores deverá orientar correções e melhorias futuras.

5.6 Processo De Manutenção

Toda alteração relevante deverá seguir:

1. Identificação Da Necessidade.
 2. Análise De Impacto.
 3. Alteração No Ambiente Development.
 4. Execução Dos Testes.
 5. Validação Das Regras De Negócio.
 6. Revisão Da Documentação.
 7. Publicação No Ambiente Live.
 8. Monitoramento Pós-Publicação.
-

6. Organização Da Interface

A aplicação utiliza páginas e elementos reutilizáveis para manter consistência visual.

As principais páginas são:

- Login.
- Signup.
- Dashboard.

- Projects.
- Project Details.
- My Tasks.
- Task Details.
- Notifications.
- Profile.

Os principais componentes reutilizáveis incluem:

- Header.
 - Sidebar.
 - Project Card.
 - Task Card.
 - Comment Card.
 - Notification Card.
 - Activity Card.
 - Dashboard Widget.
 - Empty State.
 - Confirmation Modal.
-

7. Convenções De Desenvolvimento

A aplicação utiliza nomenclatura em inglês e segue padrões definidos para facilitar manutenção.

Páginas

Utilizam `snake_case` .

Data Types

Utilizam `PascalCase` .

Campos

Utilizam `snake_case` .

Reusable Elements

Utilizam O Prefixo `re_` .

Grupos

Utilizam O Prefixo `grp_` .

Repeating Groups

Utilizam O Prefixo `rg_` .

Inputs

Utilizam O Prefixo `inp_` .

Dropdowns

Utilizam O Prefixo `ddl_` .

Date Pickers

Utilizam O Prefixo `ntp_` .

Botões

Utilizam O Prefixo `btn_` .

Ícones

Utilizam O Prefixo `ico_` .

Textos

Utilizam O Prefixo `txt_` .

Popups

Utilizam O Prefixo `pop_` .

Custom States

Utilizam O Prefixo `cs_` .

Workflows

Utilizam O Formato **Verbo + Objeto**.

Exemplos:

- Create Task.
 - Update Task.
 - Delete Task.
 - Archive Project.
 - Register Activity.
 - Mark Notification Read.
-

8. Estratégia De Testes

A validação da aplicação contempla:

- Testes Funcionais.
- Testes De Interface.
- Testes De Regras De Negócio.
- Testes De Integração.
- Testes De Privacy Rules.
- Testes De Responsividade.
- Testes De Performance.

Os testes devem ser executados antes da publicação de alterações relevantes.

9. Critérios De Qualidade

Uma funcionalidade será considerada concluída quando:

- O Requisito Estiver Implementado.
- As Validações Estiverem Funcionando.
- As Permissões Estiverem Respeitadas.
- Os Dados Estiverem Persistidos Corretamente.
- O ActivityLog Estiver Registrado Quando Aplicável.
- As Notificações Estiverem Funcionando Quando Aplicável.
- A Interface Refletir As Alterações.
- Os Testes Forem Aprovados.

- A Documentação Estiver Atualizada.
-

10. Evolução Da Solução

A arquitetura atual permite futuras expansões, como:

- Teams.
- Tags.
- Subtasks.
- Dependências Entre Tarefas.
- Checklists.
- Attachments.
- Dashboard Analítico.
- Integrações Externas.
- Aplicação Mobile.
- Quadro Kanban.
- Automações Externas.

Essas evoluções deverão ser avaliadas considerando as limitações do plano gratuito e o impacto sobre a arquitetura existente.

11. Documentação Do Projeto

A documentação oficial da solução é composta pelos seguintes artefatos:

- Planejamento Do Projeto.
- Modelagem De Dados.
- Arquitetura Da Solução.
- Guia De Desenvolvimento.
- Especificação Funcional Dos Workflows.
- Especificação Funcional Das Telas.
- Plano De Testes.
- Checklist De Publicação.
- README.

Este documento consolida as principais decisões desses artefatos e deve ser utilizado como referência geral da solução.

12. Considerações Finais

O **Ozzy - Task Manager** foi estruturado como uma solução No-Code baseada no [Bubble.io](#), utilizando os recursos nativos da plataforma para reduzir complexidade e acelerar a implementação do MVP.

A arquitetura combina:

- Um Modelo De Dados Relacional Por Referências.
- Option Sets Para Valores Controlados.
- Privacy Rules Para Controle De Acesso.
- Workflows Para Implementação Da Lógica De Negócio.
- Reusable Elements Para Padronização Da Interface.
- Ambientes Development E Live Para Controle De Publicação.
- Testes Funcionais E De Regressão Para Controle De Qualidade.
- Documentação Estruturada Para Manutenção E Evolução.

A solução foi projetada para atender aos requisitos do MVP mantendo simplicidade, baixo acoplamento e possibilidade de evolução.

Toda alteração futura deverá considerar o impacto conjunto sobre **Dados, Workflows, Privacy Rules, Interface, Testes e Documentação**, garantindo que a evolução da aplicação permaneça consistente com a arquitetura definida.

13. Artefatos Gerados

Acesse os materiais, documentos e artefatos desenvolvidos durante o planejamento e implementação do projeto.

- Link de Teste do Portal Ozzy - Task Manager, [clique aqui](#).
- Repositório Git de Artefatos, [clique aqui](#).
- Vídeo de Demonstração, [clique aqui](#).
-